

정보처리기사 필기

Engineer Information Processing — written paper (sample) · 한국산업인력공단 (Q-Net)

시험일 2026-09-19	D-DAY D-42	목표 점수 80
합격 기준 과목당 40점 이상, 전과목 평균 60점 이상	출제기준 · 버전 2026년 적용 출제기준 (샘플 · 검증 필요)	현재 자신감 ●●●○○ 3/5

이 시험은 실제로 무엇을 묻는가?

- 용어의 뜻이 아니라 유사 기술 사이의 선택 근거를 아는지
- 계층 · 포트 · 순서처럼 숫자로 갈리는 항목을 정확히 기억하는지
- 장점만 외운 수험생이 한계를 묻는 선지에서 걸리는지
- 약어를 풀어쓴 정의와 실제 동작을 연결할 수 있는지

과목 • 소프트웨어 설계 • 소프트웨어 개발 • 데이터베이스 구축 • 프로그래밍 언어 활용 • 정보시스템 구축관리	학습 자료 • 출제기준 원문 (Q-Net) • 최근 6회차 기출문제 • RFC 793 / RFC 768
---	---

학습 순환

LEARN	›	STRUCTURE	›	COMPARE	›	RETRIEVE	›	APPLY	›	CORRECT	›	COMPRESS
-------	---	-----------	---	---------	---	----------	---	-------	---	---------	---	----------

CORRECT 단계가 이 시스템의 핵심이다. 오답은 매트릭스의 특정 셀을 수정하며, 그 셀은 다음 인출 훈련에서 다시 출제된다.

셀 읽는 법 표기는 라벨 · 테두리 · 음영 세 가지로 중복 인코딩되어 흑백 인쇄에서도 유지된다.

표기	의미	무엇을 할 것인가
CORE	기본 사실	알아두면 된다. 인출 우선순위 낮음.
◆ DIST	행을 가르는 지점	한 구절로 말할 수 있어야 한다.
▲ EXC	원칙이 적용되지 않는 자리	원칙이 아니라 경계를 외운다.
■ TRAP	출제자가 오답 선지를 만드는 자리	틀린 형태와 그 이유를 함께 되된다.
⌋ UPD	개정 민감 — 시점 확인 필요	시험 전 현행 조문으로 재확인한다.
Ⓢ EVD	기출 표현 또는 출처	문장을 보는 즉시 알아본다.

영역	비중	문항 유형	우선순위	현재 자신감	최근 복습
소프트웨어 설계	20%	4지선다 20문항	B	3/5	2026-07-20
소프트웨어 개발	20%	4지선다 20문항	B	3/5	2026-07-22
데이터베이스 구축	20%	4지선다 20문항	A	2/5	2026-08-01
프로그래밍 언어 활용	20%	4지선다 20문항	A	2/5	2026-08-03
정보시스템 구축관리	20%	4지선다 20문항	A	3/5	2026-07-28

빈출 주제 - TCP / UDP 비교 - 스케줄링 알고리즘 - 정규화 단계	취약 주제 - 스케줄링 기아 현상 - 트랜잭션 격리 수준	개정 민감 주제 - 신기술 용어 (매년 신규 출제) - 보안 취약점 관련 최신 지침
--	--	---

주제 색인

#	주제	유형	우선순위	행
01	TCP vs UDP 연결 설정 여부 → 신뢰성 · 순서 · 오버헤드	IT · 정보처리	A	2
02	CPU 스케줄링 알고리즘 선점 여부 × 기아 발생 여부	IT · 정보처리	A	5

TCP vs UDP

핵심 비교축: 연결 설정 여부 → 신뢰성 · 순서 · 오버헤드

왜 중요한가

두 프로토콜의 이름은 누구나 알지만, 선지는 항상 '어느 쪽이 무엇을 보장하는가'로 갈린다. 헤더 크기와 순서 보장 여부가 특히 자주 뒤바뀌어 출제된다.

핵심 질문

이 통신 요구사항에서 신뢰성과 지연 중 무엇을 포기해도 되는가?

선행 개념 OSI 7계층 · 포트 번호 근거 RFC 793 · RFC 768

한 문장 모델

내일 시험을 보는 사람에게 설명하듯, 이 주제를 한 문장으로 쓰시오. 쓰지 못하면 아직 아는 것이 아니다.

TCP는 연결을 먼저 맺고 확인응답과 재전송으로 신뢰성과 순서를 보장하는 대신 오버헤드가 크고, UDP는 연결 없이 그냥 보내서 빠른 대신 아무것도 보장하지 않는다.

핵심 사실 다섯 가지 최대 다섯 개. 여섯 번째가 필요하다면, 그중 하나는 핵심이 아니었다.

- 1 TCP 헤더는 20바이트, UDP 헤더는 8바이트다.
- 2 순서 보장은 TCP만 한다 — UDP는 도착 순서를 보장하지 않는다.
- 3 TCP는 3-way handshake로 연결, 4-way handshake로 종료한다.
- 4 UDP도 체크섬은 있다 — '오류 검출이 전혀 없다'는 선지는 오답이다.
- 5 흐름 제어 · 혼잡 제어는 TCP만 수행한다.

CORE 핵심 DIST ◆ 변별 EXC ▲ 예외 TRAP ■ 함정 UPD ↻ 개정 EVD § 근거

직접 추가한 디테일

TCP vs UDP

결정적 변별

출제자가 단 하나의 기준으로만 이 둘을 구분하라고 한다면, 그 기준은 무엇인가?

데이터를 보내기 전에 연결을 맺는가. 맺으면 상태를 유지할 수 있으므로 확인응답 · 재전송 · 순서 · 흐름 제어가 전부 가능해지고, 맺지 않으면 전부 불가능하다.

혼동 쌍

A	B	왜 혼동되는가	결정적 단서
TCP 헤더 20바이트	UDP 헤더 8바이트	둘 다 짝수 바이트라 숫자만 외우면 뒤바뀐다.	기능이 많은 쪽이 헤더가 크다 — TCP가 20.
오류 검출	오류 복구	UDP에 체크섬이 있다는 사실 때문에 '신뢰성이 있다'로 오해한다.	UDP는 검출만 하고 버린다. 복구 (재전송)는 TCP만 한다.

예외 · 함정 매트릭스

답을 바꿀 수 있는 사실만 적는다.

진술 · 신호어	대체로 참	예외	왜 속는가	근거
■ UDP는 오류 검출 기능이 없다.	거짓	UDP 헤더에도 체크섬 필드가 있어 오류 검출은 수행한다	신뢰성이 없다는 사실을 오류 검출이 없다는 말로 확장해 버린다.	RFC 768
■ TCP는 항상 UDP보다 느리다.	참	손실률이 매우 높은 환경에서는 UDP 재전송 구현이 더 느릴 수 있다	'항상'이라는 단정어가 붙은 선지는 예외를 노린다.	일반 원리
■ TCP 연결 종료는 3-way handshake로 이루어진다.	거짓	연결은 3-way, 종료는 4-way handshake	설정 절차만 외우고 종료 절차를 같은 숫자로 답한다.	RFC 793

기출 · 근거 연결

매트릭스 셀	출처	시험 · 연도	문항	기출 표현	오답 위험
TCP · 헤더 크기	RFC 793	샘플 2025-2회	문 43	"TCP 헤더의 최소 길이는 20바이트이다"	상
UDP · 순서 보장	RFC 768	샘플 2024-3회	문 51	"순서 제어를 수행하지 않는다"	중

CPU 스케줄링 알고리즘

핵심 비교축: 선점 여부 × 기아 발생 여부

왜 중요한가

네 알고리즘의 선점 여부와 기아 발생 여부가 표 하나로 갈리는데, 매번 한 칸씩 바뀌 출제된다.

핵심 질문

선점인가 비선점인가, 그리고 기아가 발생하는가?

선행 개념 프로세스 상태 전이 근거 운영체제 개론

한 문장 모델

내일 시험을 보는 사람에게 설명하듯, 이 주제를 한 문장으로 쓰시오. 쓰지 못하면 아직 아는 것이 아니다.

스케줄링 알고리즘은 CPU를 뺏을 수 있는가(선점)와 특정 프로세스가 영원히 밀릴 수 있는가(기아)라는 두 축으로 구분된다.

핵심 사실 다섯 가지 최대 다섯 개. 여섯 번째가 필요하다면, 그중 하나는 핵심이 아니었다.

① FCFS는 비선점이며 호위 효과(convoy effect)가 발생한다.

② SJF는 평균 대기시간이 최소지만 실행시간을 미리 알 수 없다.

③ SRT는 SJF의 선점형 버전이다.

④ RR의 성능은 타임 퀀텀 크기가 결정한다.

⑤ 우선순위 스케줄링의 기아는 에이징(aging)으로 해결한다.

CPU 스케줄링 알고리즘

CPU 스케줄링 알고리즘 — 엑셀표 매트릭스						
알고리즘	◆ DIST 선점 여부	선택 기준	강점	■ TRAP 기아 발생	▲ EXC 한계	■ TRAP 시험 함정
FCFS First Come First Served	비선점	도착 순서	구현이 가장 단순, 공평	◆ 발생하지 않음	호위 효과 — 긴 작업이 앞서면 전체 지연	공평하다는 이유로 평균 대기시간도 좋다고 착각
SJF Shortest Job First	비선점	실행시간이 짧은 순	평균 대기시간 이론상 최소	발생 — 긴 작업이 계속 밀림	실행시간을 미리 알 수 없어 실제 적용이 어려움	최적이라는 서술만 보고 실현 가능하다고 판단
SRT Shortest Remaining Time	선점	남은 실행시간이 짧은 순	SJF의 응답성 개선	발생	문맥 교환 오버헤드 증가	SJF와 SRT의 선점 여부를 뒤바꾼 선지
RR Round Robin	선점	타임 퀀텀 단위 순환	응답시간이 균등, 대화형에 적합	◆ 발생하지 않음	퀀텀이 너무 작으면 문맥 교환 비용 폭증	퀀텀이 매우 커지면 FCFS와 같아진다는 점
우선순위 Priority Scheduling	선점 · 비선점 모두 가능	우선순위 값	중요 작업 우선 처리	발생 — 낮은 우선순위가 무한 대기	해결책은 에이징(aging)	기아 해결책을 '선점 전환'으로 적은 선지

CORE 핵심 DIST ◆ 변별 EXC ▲ 예외 TRAP ■ 함정 UPD ㄷ 개정 EVD § 근거

직접 추가한 디테일

CPU 스케줄링 알고리즘

결정적 변별

출제자가 단 하나의 기준으로만 이 둘을 구분하라고 한다면, 그 기준은 무엇인가?

실행 중인 프로세스에서 CPU를 뺏을 수 있는가. 뺏을 수 있으면 선점형이고 응답성이 좋아지는 대신 문맥 교환 비용이 늘어난다.

혼동 쌍

A	B	왜 혼동되는가	결정적 단서
SJF	SRT	선택 기준이 사실상 같아서 선점 여부만 다르다.	이름에 Remaining이 있으면 선점형이다.
FCFS	RR	둘 다 도착 순서를 따르므로 혼동된다.	타임 퀀텀이 언급되면 RR이다. 퀀텀이 무한대면 FCFS와 동일.

예외 · 함정 매트릭스

답을 바꿀 수 있는 사실만 적는다.

진술 · 신호어	대체로 참	예외	왜 속는가	근거
■ SJF는 항상 최적의 스케줄링이다.	참 (이론상)	실행시간을 알 수 없으므로 실제 시스템에서는 적용 불가	이론적 최적성과 실현 가능성을 구분하지 않는다.	운영체제 개론
■ 라운드 로빈에서는 기아가 발생한다.	거짓	모든 프로세스가 순환하므로 기아는 발생하지 않는다	선점형이면 기아가 생긴다는 잘못된 일반화.	운영체제 개론

기출 · 근거 연결

매트릭스 셀	출처	시험 · 연도	문항	기출 표현	오답 위험
SRT · 선점 여부	운영체제 개론	샘플 2025-1회	문 62	"SJF의 선점형 기법에 해당하는 것은?"	상

오답 기록

모든 개념 오류는 무효가 된 매트릭스 셀을 지목해야 한다.

날짜	문항	주제	내 답	정답	오류 유형	놓친 변별점	매트릭스 수정	재시험
2026-07-30	기출 2025-2회 문 43	TCP vs UDP	8바이트	20바이트	개념 혼동	기능이 많은 쪽 헤더가 크다는 연결 고리가 없었음	TCP vs UDP · TCP · 헤더 크기	2026-08- 13
2026-08-02	기출 2025-1회 문 62	스케줄링	SJF	SRT	지식 공백	Remaining이 붙으면 선점형이라는 규칙	CPU 스케줄링 알고리즘 · SRT · 선점 여부	2026-08- 16
2026-08-05	모의 2회 문 18	TCP vs UDP	오류 검출 없음	체크섬 있음	예외 망각	검출과 복구를 구분하지 않았음	TCP vs UDP · UDP · 시험 함정	2026-08- 19

· 지식 공백 · 개념 혼동 · 예외 망각 · 표현 오독 · 공식 선택 · 계산 실수 · 절차 순서 · 개정 미반영 · 단순 실수 · 시간 압박

L1 전체 매트릭스 모든 변별점, 모든 예외, 모든 근거.	L2 시험 당일 매트릭스 빈출 변별점, 예외, 공식, 함정만.	L3 마지막 10분 한 장. 압박 속에서 잊어버릴 것들.
---	---	--

시험 당일 매트릭스 — L2

표기	주제	결정적 단서
■ TRAP	헤더 크기	TCP 20바이트 / UDP 8바이트
◆ DIST	순서 보장	TCP만. UDP는 검출만 하고 복구는 안 한다.
■ TRAP	handshake	연결 3-way / 종료 4-way
◆ DIST	선점형	SRT, RR, (우선순위는 둘 다 가능)
■ TRAP	기아 발생	SJF, SRT, 우선순위 — RR과 FCFS는 발생하지 않음
CORE	기아 해결	에이징(aging)

마지막 10분 시트 — L3

- ☐ TCP 20 / UDP 8

☐ 연결 3-way, 종료 4-way

☐ UDP도 체크섬은 있다

☐ Remaining = 선점
- ☐ 퀀텀 무한대 = FCFS

☐ 기아 = SJF · SRT · 우선순위

☐ 기아 해결 = 에이징

복습 추적 날짜 또는 체크. 학습 내용보다 앞서지 않게.

주제	학습	R1	R2	R3	숙달	재점검
01 TCP vs UDP						
02 CPU 스케줄링 알고리즘						